

Name: Vispute Shreya Ganesh

Reg no.: 24BCE0563

## Operating System Lab

### Assessment 5

Implement process synchronization using semaphores / monitors

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5
#define ITEMS_PER_PRODUCER 5

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    int producer_id = *(int *)arg;

    for (int i = 1; i <= ITEMS_PER_PRODUCER; i++) {
        int item = producer_id * 100 + i;

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Producer %d produced %d at position %d\n",
            producer_id, item, in);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
```

```

        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg) {
    int consumer_id = *(int *)arg;

    for (int i = 1; i <= ITEMS_PER_PRODUCER; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumer %d consumed %d from position %d\n",
            consumer_id, item, out);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }

    return NULL;
}

int main() {
    pthread_t producer_thread;
    pthread_t consumer_thread;

    int producer_id = 1;
    int consumer_id = 1;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(
        &producer_thread,
        NULL,
        producer,
        &producer_id
    );
    pthread_create(
        &consumer_thread,
        NULL,
        consumer,
        &consumer_id
    );

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    return 0;
}

```

```

);

pthread_create(
    &consumer_thread,
    NULL,
    consumer,
    &consumer_id
);

pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);

sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

printf("Execution completed successfully.\n");

return 0;
}

```

Output:

```

shree@DESKTOP-C8NK8MS: /mnt/d/Semester 5/OSLabExp
shree@DESKTOP-C8NK8MS:/mnt/d/Semester 5/OSLabExp$ gcc Exp5.c -o Exp5 -pthread
shree@DESKTOP-C8NK8MS:/mnt/d/Semester 5/OSLabExp$ ./Exp5
Producer 1 produced 101 at position 0
Producer 1 produced 102 at position 1
Producer 1 produced 103 at position 2
Producer 1 produced 104 at position 3
Producer 1 produced 105 at position 4
Consumer 1 consumed 101 from position 0
Consumer 1 consumed 102 from position 1
Consumer 1 consumed 103 from position 2
Consumer 1 consumed 104 from position 3
Consumer 1 consumed 105 from position 4
Execution completed successfully.
shree@DESKTOP-C8NK8MS:/mnt/d/Semester 5/OSLabExp$

```

## Data Parallelism using Multithreading

Code:

```
/*
Experiment: Data Parallelism Using Multithreading
Application: Parallel Brightness Adjustment of a Grayscale Image

Each pixel has a value from 0 to 255:
0    = Black
255  = White

The image rows are divided among multiple threads.
Each thread independently increases the brightness of its assigned rows.
*/

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define MAX_ROWS 100
#define MAX_COLS 100
#define MAX_THREADS 10

int image[MAX_ROWS][MAX_COLS];
int rows, columns;
int brightnessIncrease;

typedef struct {
    int threadId;
    int startRow;
    int endRow;
} ThreadData;

void *adjustBrightness(void *argument) {
    ThreadData *data = (ThreadData *)argument;

    printf("Thread %d processing rows %d to %d\n",
           data->threadId,
           data->startRow,
           data->endRow - 1);

    for (int i = data->startRow; i < data->endRow; i++) {
```

```

        for (int j = 0; j < columns; j++) {
            image[i][j] += brightnessIncrease;

            if (image[i][j] > 255) {
                image[i][j] = 255;
            }

            if (image[i][j] < 0) {
                image[i][j] = 0;
            }
        }
    }

    pthread_exit(NULL);
}

void displayImage() {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns; j++) {
            printf("%4d", image[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int numberOfThreads;
    pthread_t threads[MAX_THREADS];
    ThreadData threadData[MAX_THREADS];

    printf("Enter number of rows in the image: ");
    scanf("%d", &rows);

    printf("Enter number of columns in the image: ");
    scanf("%d", &columns);

    if (rows <= 0 || rows > MAX_ROWS ||
        columns <= 0 || columns > MAX_COLS) {
        printf("Invalid image dimensions.\n");
        return 1;
    }
}

```

```

printf("Enter grayscale pixel values (0 to 255):\n");

for (int i = 0; i < rows; i++) {
    for (int j = 0; j < columns; j++) {
        scanf("%d", &image[i][j]);

        if (image[i][j] < 0 || image[i][j] > 255) {
            printf("Invalid pixel value at position (%d, %d).\n", i, j);
            return 1;
        }
    }
}

printf("Enter brightness adjustment value: ");
scanf("%d", &brightnessIncrease);

printf("Enter number of threads: ");
scanf("%d", &numberOfThreads);

if (numberOfThreads <= 0 || numberOfThreads > MAX_THREADS) {
    printf("Number of threads must be between 1 and %d.\n",
        MAX_THREADS);
    return 1;
}

if (numberOfThreads > rows) {
    numberOfThreads = rows;
    printf("Number of threads adjusted to %d.\n", numberOfThreads);
}

printf("\nOriginal image:\n");
displayImage();

int rowsPerThread = rows / numberOfThreads;
int remainingRows = rows % numberOfThreads;
int currentRow = 0;

for (int i = 0; i < numberOfThreads; i++) {
    int assignedRows = rowsPerThread;

    if (i < remainingRows) {
        assignedRows++;
    }
}

```

```

    }

    threadData[i].threadId = i + 1;
    threadData[i].startRow = currentRow;
    threadData[i].endRow = currentRow + assignedRows;

    currentRow = threadData[i].endRow;

    int result = pthread_create(
        &threads[i],
        NULL,
        adjustBrightness,
        &threadData[i]
    );

    if (result != 0) {
        printf("Error creating thread %d.\n", i + 1);
        return 1;
    }
}

for (int i = 0; i < numberOfThreads; i++) {
    pthread_join(threads[i], NULL);
}

printf("\nBrightness-adjusted image:\n");
displayImage();

printf("\nAll image portions were processed successfully.\n");

return 0;
}

```

Output:

```
shree@DESKTOP-C8NK8MS: /mnt/d/Semester 5/OSLabExp
shree@DESKTOP-C8NK8MS:/mnt/c/Users/shiv$ cd "/mnt/d/Semester 5/OSLabExp"
shree@DESKTOP-C8NK8MS:/mnt/d/Semester 5/OSLabExp$ gcc Exp6_2.c -o Exp6_2 -pthread
shree@DESKTOP-C8NK8MS:/mnt/d/Semester 5/OSLabExp$ ./Exp6_2
Enter number of rows in the image: 4
Enter number of columns in the image: 5
Enter grayscale pixel values (0 to 255):
10 20 30 40 50
60 70 80 90 100
110 120 130 140 150
200 210 220 230 240
Enter brightness adjustment value: 30
Enter number of threads: 2

Original image:
  10  20  30  40  50
  60  70  80  90 100
 110 120 130 140 150
 200 210 220 230 240
Thread 1 processing rows 0 to 1
Thread 2 processing rows 2 to 3

Brightness-adjusted image:
  40  50  60  70  80
  90 100 110 120 130
 140 150 160 170 180
 230 240 250 255 255

All image portions were processed successfully.
shree@DESKTOP-C8NK8MS:/mnt/d/Semester 5/OSLabExp$
```